

Development of an LLM-based 'Secret Agent' System for Resolving Picky Eating in Children

Student ID: 20243075 | Term Project #2 | Submission Date: June 18, 2026

Abstract

This report describes the results of designing and analyzing the performance of a 'Secret Agent' system utilizing Large Language Models (LLMs) to resolve the problem of picky eating in preschool children. The system consists of a Chef Agent that generates recipes to conceal ingredients and a Writer Agent that connects these recipes with the child's interests. A total of 12 prompt experiments were conducted using combinations of Learning Modes (Zero-shot / Few-shot / Contrastive Prompting) × Reasoning Techniques (CoT / Task Separation) × Verification Methods (Single-turn / Self-verification). It was confirmed that Chain-of-Thought (CoT) and Self-verification techniques play a decisive role in improving performance. Experimental results showed that the EEVE-10.8B model performed best with an average score of 85.33, and among the prompt techniques, EXP_2, which combined Zero-shot with CoT and Self-verification, recorded the highest score of 85.67. A real-time solution system was implemented by combining a Kaggle server (FastAPI + ngrok) with a Streamlit UI. In a second stage, the system was upgraded with a training-free Retrieval-Augmented Generation (RAG) pipeline that grounds recipe generation in 2,147 real Korean recipes, combining hybrid retrieval with reranking, a Corrective-RAG quality gate, and an LLM-as-a-judge grounding check. A controlled ablation over four input cases that fixed the best prompt and varied only RAG improved five of six models by an average of +10.8 points, raised dish-name concealment from 54% to 83% of cases, and reduced score variance, mainly by grounding recipes that the Baseline left vague or empty.

1. Introduction

Picky eating in children is a major cause of nutritional imbalance during growth and a serious issue that leads to conflict with parents. In particular, working mothers in their 30s who live in metropolitan areas and commute 90 minutes round-trip experience extreme pressure to prepare nutritious meals within 20 minutes after work. Conflicts that occur when a child rejects specific ingredients turn mealtime into a 'time of dread' and cause psychological helplessness and guilt for parents. The goal of this project is to utilize the creative language generation capabilities of LLMs to design 'concealment recipes' that reflect the user's refrigerator situation and the child's avoided ingredients, and to help restore family relationships by generating customized story scripts that reflect the child's interests. To this end, the effects of multi-agent structures and various prompt engineering techniques are systematically compared and analyzed. As a second-stage upgrade, this work additionally integrates recent retrieval-augmented generation techniques to ground the recipes in real recipe data, and re-evaluates the system to measure their effect.

2. Related Work

The upgrade is grounded in the retrieval-augmented generation (RAG) literature. We analyze five works spanning the origin of RAG, retrieval best practices, robustness, and evaluation, and for each we summarize the method, note its limitation, and state how it is used or adapted in our (inference-only, training-free) pipeline.

2.1 Origin of RAG (Lewis et al., 2020).

Lewis et al. introduced the RAG paradigm, coupling a parametric seq2seq generator with a non-parametric retriever (Dense Passage Retrieval over a Wikipedia index) so that generation is conditioned on retrieved passages, which reduced hallucination on knowledge-intensive tasks. A limitation for us is that their retriever and generator are jointly trained, which is costly. We keep the core idea—condition generation on retrieved evidence—but apply it to recipe generation without any training: the Chef Agent is conditioned on retrieved real

recipes instead of relying on parametric knowledge, which in the baseline produced vague or invented recipes.

2.2 Retrieval best practices (Wang et al., 2024; arXiv:2407.01219).

This empirical study compares each RAG component at scale and reports that a two-stage design—hybrid retrieval (sparse BM25 + dense embeddings) followed by a cross-encoder reranker—gives the best recall/precision trade-off, with reranking being the single most impactful component. Its limitation is that the best configuration is dataset-dependent. We adopt this configuration directly (BM25 + ko-sroberta fused by Reciprocal Rank Fusion, then a bge-reranker), targeting the baseline’s low Specificity by surfacing concrete, real recipe steps.

2.3 Corrective RAG (Yan et al., 2024; arXiv:2401.15884).

CRAG addresses a key failure mode—retrieved context that is irrelevant or wrong—by adding a retrieval evaluator that labels documents Correct, Incorrect, or Ambiguous and triggers correction when confidence is low. The original evaluator is a fine-tuned T5 model. Because our system is inference-only, we adapt the idea into a single prompt-based 1–5 relevance score on the service LLM and keep only documents above a threshold, which protects Feasibility by discarding recipes that do not fit the ingredients. In our results this gate correctly rejected the retrieval for the model that produced unusable output.

2.4 Self-RAG (Asai et al., 2024; ICLR 2024, arXiv:2310.11511).

Self-RAG trains a model to emit reflection tokens that decide when to retrieve and critique whether its own output is supported by evidence (on-demand retrieval plus self-critique). We considered this for the verification stage but did not adopt its self-reflection framing: our baseline already uses a self-verification loop (EXP_2), so an added self-critique would be mechanistically redundant. This trade-off motivated our choice of an external judge instead, described next.

2.5 Reference-free RAG evaluation (Sardana, 2025; arXiv:2503.21157).

This work benchmarks reference-free methods—needing no ground-truth labels—for detecting hallucinations in RAG outputs (LLM-as-a-judge, Lynx, HHEM, and others), finding that several detect incorrect responses with high precision. We adopt the LLM-as-a-judge variant as a final grounding check that scores (0–25) whether the generated recipe is supported by the retrieved recipes. Being an external judge applied after generation, it is distinct from the baseline self-verification loop and adds a faithfulness signal without redundancy.

3. Methods

3.1 System Configuration and Tech Stack

This project is divided into an experimental evaluation pipeline and a real-time service system.

Real-time Service System Configuration

- Frontend: Streamlit — Input of user information (fridge ingredients, disliked foods, interests) and visualization of results.
- Backend: FastAPI (app.py, llmservice.py) — LLM inference server, provides /generate endpoint.
- Infrastructure: Kaggle Notebook (GPU resource utilization), ngrok (external communication tunneling for local-Kaggle connection).
- Service Model: EEVE-10.8B (BF16 quantization, automatic GPU distribution).

Experimental Evaluation Pipeline

- Experimental Models: EEVE-10.8B, ko-gemma-2-9b, llama-3-Blossom-8B, Trillion-7B, Llama-3-Alpha-8B, SOLAR-10.7B.
- Evaluator: Gemini model — Automatically scores generated results using quantitative indicators.
- Prompt Implementation: recipe_prompts.py (Recipe Agent), story_prompts.py (Story Agent).
- GitHub Repository: https://github.com/topop0628/SSU_term_project-1 (if private, “ssuai” is invited as a collaborator).

3.2 Prompt Engineering Experimental Design

To systematically analyze the impact of prompt techniques, 12 experiments were designed using combinations of three axes: Learning Mode × Reasoning Technique × Verification Method.

Exp	Learning Mode	Reasoning Technique	Verification Method	Total
EXP_1	Zero-shot	CoT (Step-by-step)	Single-turn	68.17
EXP_2	Zero-shot	CoT (Step-by-step)	Self-verification (Max 3)	85.67
EXP_3	Zero-shot	Task Separation	Single-turn	58.17
EXP_4	Zero-shot	Task Separation	Self-verification (Max 3)	73.17
EXP_5	One-shot	CoT (Step-by-step)	Single-turn	74.17
EXP_6	One-shot	CoT (Step-by-step)	Self-verification (Max 3)	77.83
EXP_7	One-shot	Task Separation	Single-turn	77.17
EXP_8	One-shot	Task Separation	Self-verification (Max 3)	69.50
EXP_9	Contrastive	CoT (Step-by-step)	Single-turn	85.33
EXP_10	Contrastive	Task Separation	Self-verification (Max 3)	69.50
EXP_11	Contrastive	CoT (Step-by-step)	Self-verification (Max 3)	78.67
EXP_12	Contrastive	Task Separation	Self-verification (Max 3)	79.83

Table 1. Experimental Design Matrix.

Definitions of each axis:

- **Learning Mode** — Zero-shot: No examples / Few-shot: 1 correct example / Contrastive: Success/failure contrast examples.
- **Reasoning Technique** — CoT: ‘Think step-by-step’ instruction / Task Separation: Strategy → Plan → Generation.
- **Verification Method** — Single-turn: Finish after 1 generation / Self-verification: Up to 3 self-correction loops.

3.3 Analysis and Evaluation Methods

Results were quantitatively scored using the Gemini model as a judge.

- **Chef Agent (Recipe Agent):** Concealment, Feasibility, Specificity, JSON Format (25 points each, total 100).
- **Writer Agent (Story Agent):** Interest Connectivity, Child-friendliness, Interest Induction, Conciseness (25 points each, total 100).

3.4 Retrieval-Augmented Generation Pipeline (Upgrade)

The upgrade grounds the Chef Agent in real recipes. The recipe database is built from the KoreanRecipeGPT corpus: each entry is parsed into a schema of {dish_name, ingredients, steps} and deduplicated, yielding 2,147 recipes (e.g., 285 contain carrot, 279 egg, 216 cheese). The RAG pipeline has four stages, all at inference time:

- **Retrieve (Wang et al., 2024):** hybrid search combining BM25 and dense embeddings (ko-sroberta-multitask) fused by Reciprocal Rank Fusion, then a cross-encoder reranker (bge-reranker-v2-m3) selects the top-3 recipes.
- **Filter (CRAG, Yan et al., 2024):** a prompt-based evaluator scores each candidate 1–5 and keeps only those above a threshold (labels Correct / Ambiguous / Incorrect).
- **Ground:** the surviving recipes are injected as reference context, with a guard instructing the model not to copy a dish name containing the disliked ingredient. This block is added in the RAG condition only, so the Baseline prompt stays unchanged.
- **Verify (Sardana, 2025):** after generation, an external LLM-as-a-judge scores 0–25 whether the recipe is grounded in the retrieved evidence.

3.5 RAG Experiment Design

To isolate the effect of RAG, the best baseline prompt (EXP_2) is fixed and only RAG on/off is varied across all six models (controlled ablation). To test generalization, four input cases are used (disliked ingredient in parentheses): carrot, egg, cheese (carrot); spinach, tofu, egg (spinach); broccoli, potato, cheese (broccoli); eggplant, egg, onion (eggplant), each with a different interest. Decoding is deterministic, so the Baseline reproduces the original system. Each model runs in its own subprocess on a single NVIDIA A100 so that GPU memory is fully released between models. Both the Recipe Agent (Concealment, Feasibility, Specificity, JSON Format) and the Story Agent (Interest Connectivity, Child-friendliness, Interest Induction, Conciseness) are scored on the four-criteria rubric (each 0–25, total 100). The Story Agent does not use RAG; because RAG changes the dish name, the two conditions are still compared to measure the downstream effect.

4. Results

4.1 Performance Evaluation Results by Model

Model	Average Total Score
EEVE-10.8B	85.33
ko-gemma-2-9b	82.33
llama-3-Blossom-8B	78.83
Trillion-7B	78.00
Llama-3-Alpha-8B	72.67
SOLAR-10.7B	51.42

Table 2. Comparison of Average Total Scores by Model.

EEVE-10.8B ranked first with 85.33 points, while SOLAR-10.7B was last at 51.42. This suggests that a model's Korean instruction-following ability significantly influences performance.

4.2 Detailed Results by Experiment (Recipe Agent, EEVE-10.8B)

Exp	Concealment	Feasibility	Specificity	JSON Format	Total
EXP_1	15.00	20.83	12.33	20.00	68.17

Exp	Concealment	Feasibility	Specificity	JSON Format	Total
EXP_2	21.67	25.00	14.83	24.17	85.67
EXP_3	12.50	19.17	10.67	15.83	58.17
EXP_4	19.17	20.83	13.17	20.00	73.17
EXP_5	19.17	19.17	10.83	25.00	74.17
EXP_6	22.50	21.17	10.00	24.17	77.83
EXP_7	17.50	23.33	12.17	24.17	77.17
EXP_8	18.33	20.50	10.67	20.00	69.50
EXP_9	23.33	25.00	12.83	24.17	85.33
EXP_10	19.17	19.17	10.33	20.83	69.50
EXP_11	23.33	19.17	11.17	25.00	78.67
EXP_12	22.50	20.83	11.50	25.00	79.83

Table 3. Detailed Evaluation Scores of Recipe Agent by Experiment.

4.3 RAG Ablation Results (Baseline vs. RAG, four cases)

Table 4 reports the Recipe Agent total (0–100) for each model and condition across the four cases, with the mean and standard deviation (SD). RAG raised the mean for five of six models and, notably, reduced the SD for most—making the output more consistent—because the Baseline collapsed on harder cases (broccoli, eggplant) with empty content, while RAG produced grounded recipes.

Model	Cond.	C1	C2	C3	C4	Mean	SD	Δ
EEVE-10.8B	Base	81	71	41	31	56.0	20.6	
	RAG	83	76	88	85	83.0	4.4	+27.0
ko-gemma-2-9b	Base	83	84	80	84	82.8	1.6	
	RAG	87	84	84	84	84.8	1.3	+2.0
Bllossom-8B	Base	66	77	74	75	73.0	4.2	
	RAG	85	82	83	45	73.8	16.6	+0.8
Trillion-7B	Base	74	85	73	86	79.5	6.0	
	RAG	88	87	86	85	86.5	1.1	+7.0
SOLAR-10.7B	Base	62	41	62	46	52.8	9.5	
	RAG	74	73	63	69	69.8	4.3	+17.0
Llama-3-Alpha-8B	Base	25	22	26	64	34.3	18.0	
	RAG	25	22	26	64	34.3	18.0	0.0

Table 4. Recipe Agent total per case (C1–C4), mean, SD, and change in mean (EXP_2 fixed).

Group	Baseline	RAG	Δ
All 6 models	63.0	72.0	+9.0
Excluding Llama-3-Alpha (5 models)	68.8	79.6	+10.8

Table 5. Recipe Agent mean total over four cases, Baseline vs. RAG.

4.4 Dish-Name Concealment and Story Agent

Table 6 counts the cases (out of four) in which the dish name did *not* expose the disliked ingredient. Concealment success rose from 13/24 (54%) to 20/24 (83%).

Model	Baseline	RAG
EEVE-10.8B	2 / 4	3 / 4
ko-gemma-2-9b	4 / 4	4 / 4
llama-3-Blossom-8B	3 / 4	4 / 4
Trillion-7B	2 / 4	4 / 4
SOLAR-10.7B	0 / 4	3 / 4
Llama-3-Alpha-8B	2 / 4	2 / 4
Total	13 / 24 (54%)	20 / 24 (83%)

Table 6. Dish-name concealment success (cases with no ingredient leak in the title).

Table 7 reports the Story Agent mean total. The Story Agent does not use RAG; the small change confirms that RAG's effect is concentrated in the recipe, propagating only weakly to the story (mainly EEVE, whose Baseline story for the eggplant case was highly repetitive).

Model	Baseline	RAG	Δ
EEVE-10.8B	77.5	85.0	+7.5
ko-gemma-2-9b	82.8	83.5	+0.7
llama-3-Blossom-8B	83.3	81.5	-1.8
Trillion-7B	85.3	85.3	0.0
SOLAR-10.7B	33.3	34.3	+1.0
Llama-3-Alpha-8B	5.0	5.0	0.0
Overall	61.2	62.4	+1.2

Table 7. Story Agent mean total over four cases (downstream effect; Story Agent itself has no RAG).

5. Discussion

Experimental results provided three main implications:

- **First**, the effect of Self-verification was most prominent. Comparing EXP_1 and EXP_2, scores increased by +17.5 points with just one verification loop. This shows that self-correction significantly improves output quality.
- **Second**, Contrastive Prompting was effective in providing clear standards. EXP_9 achieved the highest score in concealment (23.33).
- **Third**, Task Separation showed lower scores due to context continuity loss, resulting in lower JSON format compliance.

5.4 Comparison of Actual Output Examples (EEVE-10.8B, Input: Carrot/Egg/Cheese)

The table below compares the actual output results for the same input (Ingredients: Carrot, Egg, Cheese / Disliked: Carrot / Interest: Space).

구분	EXP_2 (85.67점 — 최고)	EXP_3 (58.17점 — 최저)
프롬프트 기법	Zero-shot + CoT + 자가검토(최대 3회)	Zero-shot + 작업 분리 + 단판 생성
[Recipe] 전략	당근 맛·향·식감을 숨기기 위해 잘게 썬 당근을 계란과 치즈와 함께 볶음밥에 섞어주세요.	당근의 맛·향·식감을 완벽하게 은폐하기 위해, 당근을 잘게 썰어 계란과 치즈와 함께 볶음밥에 섞어주세요.
[Recipe] 조리법	1. 당근을 잘게 썰어주세요. 2. 계란과 치즈를 섞어주세요. 3. 팬에 기름을 두르고 볶기	1. 당근을 잘게 썰어주세요. 2. 계란과 치즈를 섞어주세요. 3. 팬에 기름을 두르고 볶기
[Story] 전략	우주에서 온 치즈 계란 볶음밥	우주에서 온 치즈 계란 볶음밥을 상상해 보세요.
[Story] 동화 문장	우주에서 온 특별한 치즈 계란 볶음밥을 맛보세요!	우주선에서 온 치즈 계란 볶음밥을 맛보세요!
JSON 형식 준수	O 정상 (24.17/25)	X 저조 (15.83/25)

Table 8. Comparison of actual outputs from EXP_2 (Best) and EXP_3 (Worst).

5.5 Effect of Retrieval-Augmented Generation

Across the four cases, the RAG ablation shows three effects. First, dish-name concealment improved (54%→83%): in the Baseline, Trillion produced “Sweet Carrot Omelette” and SOLAR produced “Hidden Carrot Cheesy Egg Bites,” both exposing the ingredient, while RAG produced “Rainbow Fried Rice” and “Hidden Vegetable Omelette.” Second, RAG both raised the mean and reduced the variance: the Baseline collapsed on harder cases (e.g., EEVE returned empty content for broccoli and eggplant, scoring 41 and 31), whereas RAG grounded these into full recipes, lifting EEVE from 56.0±20.6 to 83.0±4.4. Third, the gains are largest where the Baseline was weakest (EEVE +27.0, SOLAR +17.0), and small where it was already strong and clean (ko-gemma +2.0).

The results are reported honestly. RAG does not always help: Bllossom returned empty content on the eggplant case under RAG (score 45), so its variance actually rose (16.6) and its mean barely moved (+0.8). The title guard is also imperfect—EEVE (“spinach hidden fried rice”) and SOLAR (“Hidden Broccoli Delight”) still leaked the ingredient in one case each. Llama-3-Alpha produced empty or identical output in both conditions, a model failure unrelated to RAG. The Story Agent, which has no RAG, changed little (+1.2 overall); only EEVE improved (+7.5) because RAG removed a highly repetitive baseline story. The whole pipeline is training-free, adding only one short LLM call for the gate and one for the grounding judge.

6. Conclusion

This study experimentally proved that an LLM-based multi-agent system can be a useful tool for resolving children’s picky eating. Key achievements include generating ingredient-concealing recipes and creatively linking stories to dishes. The core message is that ‘the combination of CoT and self-verification maximizes model performance’. In the second stage, a training-free RAG pipeline was added to ground recipes in real recipe data; over four input cases it improved five of six models by an average of +10.8 points, raised dish-name concealment from 54% to 83%, and made the outputs more consistent (lower variance), mainly by grounding recipes that the Baseline left vague or empty. Future research plans include scaling the recipe database (e.g., AI-Hub Korean cooking data), adding a multimodal storybook illustration agent, and conducting actual user tests with parents and children.

7. References

[1] Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.
[2] Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS.
[3] Madaan, A. et al. (2023). Self-Refine: Iterative Self-Correction with LLMs. arXiv.
[4] Google. (2024). Gemma: Open Models Based on Gemini Technology. DeepMind.
[5] Streamlit Inc. (2024). Streamlit Documentation.

- [6] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- [7] Wang, X. et al. (2024). Searching for Best Practices in Retrieval-Augmented Generation. arXiv:2407.01219.
- [8] Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. arXiv:2401.15884.
- [9] Asai, A. et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR. arXiv:2310.11511.
- [10] Sardana, A. (2025). Real-Time Evaluation Models for RAG: Who Detects Hallucinations Best? arXiv:2503.21157.
- [11] KoreanRecipeGPT dataset (skku-taehwan). GitHub repository.

8. Actual UI

